



Estrutura de Dados e Algoritmos

EDA

Docentes: José Nhavoto
Yacub Mussá

Índice

1 Recursividade

1.1 O que é Recursividade?

Recursividade é uma técnica de programação na qual uma função chama a si própria, directa ou indirectamente, para resolver um problema. Cada chamada recursiva resolve uma subparte do problema, até chegar a um caso base — uma condição simples o suficiente para ser resolvida directamente, sem novas chamadas recursivas.

Elementos Fundamentais da Recursividade:

1. **Caso Base:**

O momento onde a função não faz mais chamadas a si mesma e retorna um resultado simples.

2. **Chamada Recursiva:**

A função chama a si própria, geralmente com parâmetros menores ou mais simples, aproximando-se do caso base a cada chamada.

3. **Progresso em Direção ao Caso Base:**

Cada chamada recursiva deve modificar seu estado para eventualmente alcançar o caso base.

1.2 Fluxo Básico de uma Função Recursiva

1. Verificar se o caso base foi alcançado.
2. Se não, chamar a função novamente com novos parâmetros (mais simples).
3. Continuar até o caso base ser atingido.
4. Retornar os resultados das chamadas recursivas até a chamada original.

1.3 Por que Usar a Recursividade?

- Permite resolver problemas que têm subproblemas similares ao problema original, como algoritmos de busca, ordenação, estruturas hierárquicas (árvores), programação dinâmica, entre outros.
- Em muitos casos, soluciona problemas com menos código, tornando o algoritmo mais legível.
- Facilita a compreensão conceitual de algoritmos complexos, como divisão e conquista.

1.4 Atenção com a Recursividade

- **Stack Overflow:** Quando a recursão é muito profunda e ultrapassa o limite de chamadas de função da linguagem, ocorre um erro.
- **Falta de Caso Base:** Se a função nunca chegar ao caso base, entra em loop infinito.
- **Consumo de Memória:** Recursão pode ser menos eficiente em consumo de memória por empilhar chamadas.

1.5 Exemplos Práticos Explicados

Exemplo 1: Cálculo de Factorial

Descrição:

O fatorial de um número n , representado por $n!$, é a multiplicação de todos os naturais de 1 até n .

Passo a Passo

1. **Identificar o caso base:** $0! = 1$
2. **Chamada recursiva:** $n! = n * (n - 1)!$
3. **Implementar em Java:**

```
public int fatorial(int n) {  
    if (n == 0) {  
        return 1; // Caso base  
    } else {  
        return n * fatorial(n - 1); // Chamada recursiva  
    }  
}
```

Exemplo de execução:

$\text{fatorial}(4) \rightarrow 4 * \text{fatorial}(3)$

$\text{fatorial}(3) \rightarrow 3 * \text{fatorial}(2)$

$\text{fatorial}(2) \rightarrow 2 * \text{fatorial}(1)$

$\text{fatorial}(1) \rightarrow 1 * \text{fatorial}(0)$

$\text{fatorial}(0) \rightarrow 1$ (caso base)

Resultado: $4 * 3 * 2 * 1 * 1 = 24$

Exemplo 2: Fibonacci

Descrição:

Sequência de Fibonacci: cada número é a soma dos dois anteriores.

$f(0) = 0;$

$f(1) = 1;$

$f(n) = f(n-1) + f(n-2)$ para $n > 1$.

Passo a Passo

1. **Identificar casos base:** $f(0) = 0, f(1) = 1$
2. **Chamada recursiva:** $f(n) = f(n-1) + f(n-2)$
3. **Implementar em Java:**

```

public int fibonacci(int n) {
    if (n == 0) {
        return 0;
    }
    if (n == 1) {
        return 1;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}

```

Exemplo de execução:

$\text{fibonacci}(4) \rightarrow \text{fibonacci}(3) + \text{fibonacci}(2)$

$\text{fibonacci}(3) \rightarrow \text{fibonacci}(2) + \text{fibonacci}(1)$

$\text{fibonacci}(2) \rightarrow \text{fibonacci}(1) + \text{fibonacci}(0)$

Exemplo 3: Soma dos Elementos de um Vetor

Descrição:

Calcular a soma dos elementos de um vetor usando recursividade.

Passo a Passo

1. **Caso base:** Quando o vetor tem tamanho zero, a soma é 0.
2. **Chamada recursiva:** $\text{somaVetor}(\text{vetor}, n) = \text{vetor}[n-1] + \text{somaVetor}(\text{vetor}, n-1)$
3. **Implementação em Java:**

```

public int somaVetor(int[] vetor, int n) {
    if (n == 0) {
        return 0;
    } else {
        return vetor[n - 1] + somaVetor(vetor, n - 1);
    }
}

```

Exemplo de execução:

Vector=[6, 4, 2, 0]

$\text{somaVetor}(, 3) \rightarrow 6 + \text{somaVetor}(, 2)$

$\rightarrow 4 + \text{somaVetor}(, 1)$

$\rightarrow 2 + \text{somaVetor}(, 0)$

$\rightarrow 0$

Resposta: $6 + 4 + 2 + 0 = 12$

Exemplo 4: Inverter uma String

Descrição:

Dada uma string, retornar ela invertida usando recursão.

Passo a Passo

1. **Caso base:** String vazia ou de tamanho 1, retorna a própria string.
2. **Chamada recursiva:** `inverter(s) = inverter(s.substring(1)) + s.charAt(0)`
3. **Implementação em Java:**

```
public String inverter(String s) {  
    if (s.length() <= 1) {  
        return s;  
    }  
    return inverter(s.substring(1)) + s.charAt(0);  
}
```

Exemplo de execução:

`inverter("abc")` → `inverter("bc") + "a"`
→ `inverter("c") + "b" + "a"`
→ `"c" + "b" + "a" = "cba"`

Ficha de Exercícios Número 1

1. Crie uma função recursiva para calcular a soma dos números de 1 a n.
2. Crie uma função recursiva para verificar se uma string é um palíndromo (lê igual de trás para frente).
3. Implemente uma função recursiva que conta o número de dígitos de um número inteiro positivo.
4. Implemente uma função recursiva que procura um valor em um vetor ordenado (busca binária).
5. Construa uma função recursiva para calcular o valor de um número elevado a uma potência (exponenciação).
6. Desenvolva uma função recursiva que imprime todos os números naturais de n até 1.
7. Escreva uma função recursiva que conte o número de elementos pares em um vetor de inteiros.
8. Escreva uma função recursiva que retorne o maior valor de um vetor de inteiros.
9. Crie uma função recursiva que encontra o mínimo divisor de um número maior que 1.
10. Implemente uma função recursiva para calcular a soma dos dígitos de um número inteiro.